JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class AtomicIntegerArray

java.lang.Object
    java.util.concurrent.atomic.AtomicIntegerArray

All Implemented Interfaces:

Serializable

```
public class AtomicIntegerArray
extends Object
implements Serializable
```

An `int` array in which elements may be updated atomically. See the `java.util.concurrent.atomic` package specification for description of the properties of atomic variables.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`AtomicIntegerArray(int length)`	Creates a new AtomicIntegerArray of the given length, with all elements initially zero.
`AtomicIntegerArray(int[] array)`	Creates a new AtomicIntegerArray with the same length as, and all

	elements copied from, the given array.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
int	**accumulateAndGet**(int i, int x, **IntBinaryOperator** accumulatorFunction)	Atomically updates the element at index i with the results of applying the given function to the current and given values, returning the updated value.
int	**addAndGet**(int i, int delta)	Atomically adds the given value to the element at index i.
boolean	**compareAndSet**(int i, int expect, int update)	Atomically sets the element at position i to the given updated value if the current value == the expected value.
int	**decrementAndGet**(int i)	Atomically decrements by one the element at index i.
int	**get**(int i)	

		Gets the current value at position i.
int	**getAndAccumulate**(int i, int x, **IntBinaryOperator** accumulatorFunction)	Atomically updates the element at index i with the results of applying the given function to the current and given values, returning the previous value.
int	**getAndAdd**(int i, int delta)	Atomically adds the given value to the element at index i.
int	**getAndDecrement**(int i)	Atomically decrements by one the element at index i.
int	**getAndIncrement**(int i)	Atomically increments by one the element at index i.
int	**getAndSet**(int i, int newValue)	Atomically sets the element at position i to the given value and returns the old value.
int	**getAndUpdate**(int i, **IntUnaryOperator** updateFunction)	Atomically updates the element at index i with

		the results of applying the given function, returning the previous value.
int	**incrementAndGet**(int i)	Atomically increments by one the element at index i.
void	**lazySet**(int i, int newValue)	Eventually sets the element at position i to the given value.
int	**length**()	Returns the length of the array.
void	**set**(int i, int newValue)	Sets the element at position i to the given value.
String	**toString**()	Returns the String representation of the current values of array.
int	**updateAndGet**(int i, **IntUnaryOperator** updateFunction)	Atomically updates the element at index i with the results of applying the given function, returning the updated value.

boolean	**weakCompareAndSet**(int i, int expect, int update)	Atomically sets the element at position i to the given updated value if the current value == the expected value.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

AtomicIntegerArray

```
public AtomicIntegerArray(int length)
```

Creates a new AtomicIntegerArray of the given length, with all elements initially zero.

Parameters:

`length - the length of the array`

AtomicIntegerArray

```
public AtomicIntegerArray(int[] array)
```

Creates a new AtomicIntegerArray with the same length as, and all elements copied from, the given array.

Parameters:

`array - the array to copy elements from`

Throws:

NullPointerException - if array is null

Method Detail

length

```
public final int length()
```

Returns the length of the array.

Returns:

the length of the array

get

```
public final int get(int i)
```

Gets the current value at position `i`.

Parameters:

`i` - the index

Returns:

the current value

set

```
public final void set(int i,
                      int newValue)
```

Sets the element at position `i` to the given value.

Parameters:

`i` - the index

`newValue` - the new value

lazySet

```
public final void lazySet(int i,
                          int newValue)
```

Eventually sets the element at position `i` to the given value.

Parameters:

`i` - the index

`newValue` - the new value

Since:

1.6

getAndSet

```
public final int getAndSet(int i,
                           int newValue)
```

Atomically sets the element at position `i` to the given value and returns the old value.

Parameters:

`i - the index`

`newValue - the new value`

Returns:

`the previous value`

compareAndSet

```
public final boolean compareAndSet(int i,
                                   int expect,
                                   int update)
```

Atomically sets the element at position `i` to the given updated value if the current value == the expected value.

Parameters:

`i - the index`

`expect - the expected value`

`update - the new value`

Returns:

`true if successful. False return indicates that the actual value was not equal to the expected value.`

weakCompareAndSet

```
public final boolean weakCompareAndSet(int i,
                                       int expect,
                                       int update)
```

Atomically sets the element at position `i` to the given updated value if the current value == the expected value.

May fail spuriously and does not provide ordering guarantees, so is only rarely an appropriate alternative to `compareAndSet`.

Parameters:

`i - the index`

`expect - the expected value`

`update - the new value`

Returns:

`true if successful`

getAndIncrement

```
public final int getAndIncrement(int i)
```

Atomically increments by one the element at index `i`.

Parameters:

`i - the index`

Returns:

`the previous value`

getAndDecrement

```
public final int getAndDecrement(int i)
```

Atomically decrements by one the element at index `i`.

Parameters:

`i - the index`

Returns:

`the previous value`

getAndAdd

```
public final int getAndAdd(int i,
                           int delta)
```

Atomically adds the given value to the element at index `i`.

Parameters:

`i - the index`

`delta - the value to add`

Returns:

`the previous value`

incrementAndGet

```
public final int incrementAndGet(int i)
```

Atomically increments by one the element at index `i`.

Parameters:

`i - the index`

Returns:

`the updated value`

decrementAndGet

```
public final int decrementAndGet(int i)
```

Atomically decrements by one the element at index `i`.

Parameters:

`i - the index`

Returns:

`the updated value`

addAndGet

```
public final int addAndGet(int i,
                           int delta)
```

Atomically adds the given value to the element at index `i`.

Parameters:

`i - the index`

`delta - the value to add`

Returns:

`the updated value`

getAndUpdate

```
public final int getAndUpdate(int i,
                              IntUnaryOperator updateFunction)
```

Atomically updates the element at index `i` with the results of applying the given function, returning the previous value. The function should be side-

effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i - the index`

`updateFunction - a side-effect-free function`

Returns:

`the previous value`

Since:

`1.8`

updateAndGet

```
public final int updateAndGet(int i,
                              IntUnaryOperator updateFunction)
```

Atomically updates the element at index `i` with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i - the index`

`updateFunction - a side-effect-free function`

Returns:

`the updated value`

Since:

`1.8`

getAndAccumulate

```
public final int getAndAccumulate(int i,
                                  int x,
                                  IntBinaryOperator accumulatorFunction)
```

Atomically updates the element at index `i` with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i - the index`

x - the update value

accumulatorFunction - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final int accumulateAndGet(int i,
                                  int x,
                                  IntBinaryOperator accumulatorFunction)
```

Atomically updates the element at index `i` with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value at index `i` as its first argument, and the given update as the second argument.

Parameters:

i - the index

x - the update value

accumulatorFunction - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

toString

```
public String toString()
```

Returns the String representation of the current values of array.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current values of array

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.